DevAtlas | MVP Product Brief

20 October 2026 | Stay current. Understand the tools. Connect the stack. Build better.

Vision

Become the practical knowledge layer for software development - continuously updated, interconnected, and focused on helping developers build.

Mission

Help developers stay current, understand, choose, connect, and practically use modern development technologies without piecing together information from dozens of sources.

MVP questions answered

Who is it for? Beginner/intermediate developers learning to integrate technologies and working developers tracking releases.

What problem does it solve? Fragmented documentation, unclear stack choices, difficult integrations and overwhelming release notes.

What is the differentiator? Source-backed release explanations linked to tool, integration and practical build guides.

What is free? Core content, release explanations, stack and build guides.

What technologies launch? JavaScript, TypeScript, React, Next.js, Node.js, Express, PostgreSQL, MongoDB, Git, Docker, Vitest, Socket.IO.

How much content? 12 tool pages, up to 8 verified integration guides, 3 build guides and a curated set of sourced release breakdowns.

What is on each tool page? Purpose, use cases, when not to use, setup, concepts, examples, related tools, official sources, current version and verification dates.

What does an update include? Important changes, why they matter, working examples, breaking changes, upgrade considerations, original sources.

How are updates published? Official-source checks and human-reviewed drafts; no unsupported automated publication.

What does a stack guide include? Architecture, prerequisites, installation, code, explanation, testing, troubleshooting and production notes.

What search is supported? Tools, stack combinations and guide titles with category filters and useful empty states.

Does V1 require user accounts? No public accounts. Secure private admin only.

Is AI included? No public AI assistant; optional editorial assistance requires source verification.

How is quality maintained? Source links, version applicability, editorial review, last-verified timestamps and correction workflow.

What defines success? Useful search outcomes, guide engagement, working project completions, return visits and actionable feedback.

What is excluded? Social features, marketplace, payments, mobile app, massive catalog, autonomous AI publishing.

Technology decisions

Next.js App Router, React, strict TypeScript, Tailwind CSS, React Icons, MongoDB Atlas and Mongoose, Zod, modular Route Handlers/services/repositories, Vitest, Playwright, Vercel. Validate current dependency compatibility before installation.

Initial API contracts to define in TRD

GET /api/tools, /api/tools/[slug], /api/updates, /api/updates/[slug], /api/stacks, /api/stacks/[slug], /api/builds, /api/builds/[slug], /api/search. Admin routes require authentication and authorization.

External credentials

MongoDB Atlas URI required for real database integration; deployment credentials required for release. GitHub API token optional if official release monitoring needs higher quotas. No email provider or paid AI API needed in V1.

Vertical-slice implementation phases

0. Verify repository boundary, project conventions and six planning documents.

1. Foundation: application shell, database connection, health checks and responsive UI.

2. Tool catalog: schemas, seeds, APIs, listing/search UI and detail pages.

3. Release updates: official source adapters, drafts, review, APIs and What's New feed.

4. Stack guides: relationships, APIs, architecture visuals and integration pages.

5. Build guides: project content, code samples, APIs and tutorial experience.

6. Admin: secure access, content editing, publishing and audit logs.

7. Discovery: improved search, filters, related links and privacy-aware analytics.

8. Launch: security, accessibility, SEO, E2E, performance, deployment and feedback.

Documentation

Use the existing Codex six-Markdown-file skill. Suggested documents: prd.md, trd.md, appflow.md, design-brief.md, backend-schema.md, implementation-plan.md. Respect the skill's canonical naming.

Per-phase definition of done

Every phase ships working backend, API and frontend together, with loading/error states, tests, docs, responsive checks and an explicit list of required external services. Commit only after checks pass.

Before public release

Verify brand/domain/trademark availability, official data sources and their reuse terms, live deployment configuration, content accuracy and feedback from real developers.